

ESCUELA DE
INGENIERÍA INFORMÁTICA



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

Sistema de Gestión de Reservas en Parques Nacionales

MATÍAS ANDRADE

JOAQUIN PIZARRO

LUCIANO CASTAÑEDA

Asignatura: Programación Avanzada

OCTUBRE 2026

ÍNDICE

ÍNDICE	2
1. Introducción	3
2. Requerimientos del Proyecto	4
2.1 SIA-1 — Análisis de datos y funcionalidades	4
2.2 SIA-2 — Diseño conceptual de clases y código Java	5
2.3 SIA-3 — Buenas prácticas	5
2.4 SIA-4 — Colecciones de objetos	6
2.5 SIA-5 — Sobrecarga de métodos	7
Primera clase: Usuario	7
Segunda clase: Tarifa	8
2.6 SIA-6 — Sobreescritura de métodos	8
2.7 SIA-7 — Inserción y listado	9
Inserción de usuarios	9
Listado de usuarios	10
Inserción de reservas	10
Listado de reservas	11
2.8 SIA-8 — Edición, eliminación y búsqueda	11
Usuarios	11
Reservas	12
2.9 SIA-9 — Funcionalidad propia de utilidad para el negocio	12
2.10 SIA-10 — Consola y ventanas	13
2.11 SIA-11 — Persistencia de datos	14
2.12 SIA-12 — Excepciones personalizadas	16
CapacidadExcedidaException	16
ReservaNoEncontradaException	16
2.13 SIA-13 — GitHub	17
2.14 SIA-O3 — Documentación mediante Javadoc	18
3. Conclusión	20
4. Anexos	21
Diagrama UML completo	21
Estructura de carpetas	22
Archivos de datos utilizados	23

1. Introducción

El proyecto corresponde a un sistema de gestión de reservas para un parque, que permite administrar los recursos disponibles, los usuarios y las reservas realizadas. El sistema contempla distintos tipos de recursos, como cabañas, zonas de camping y actividades, además de información asociada a tarifas y permisos.

El sistema permite realizar operaciones sobre usuarios, recursos y reservas, incluyendo inserción, búsqueda, modificación, eliminación, listado y gestión de reservas.

Además, el proyecto cuenta con dos modalidades de interacción: una interfaz mediante consola y una interfaz gráfica mediante ventanas.

El objetivo del proyecto es desarrollar un sistema orientado a objetos que permita gestionar los recursos de un parque y las reservas realizadas por sus usuarios, aplicando conceptos de programación orientada a objetos, colecciones, persistencia de datos, interfaces gráficas y manejo de información.

El sistema busca solucionar la necesidad de administrar de manera organizada los recursos de un parque y las reservas asociadas a ellos. Permite mantener información de usuarios, recursos y reservas, facilitando la consulta de disponibilidad, creación y cancelación de reservas, modificación de información y gestión de permisos.

Las tecnologías utilizadas para el desarrollo del proyecto fueron Java como lenguaje de programación, aplicando los principios de la Programación Orientada a Objetos y utilizando principalmente colecciones Map de Java. Para la interfaz gráfica se utilizó Swing, complementada con una interfaz mediante consola. Además, se utilizaron archivos para la persistencia de la información. Para el diseño y modelado de las clases se empleó UML, utilizando Visual Paradigm para la elaboración del diagrama de clases. Finalmente, se utilizó GitHub como sistema de control de versiones y gestión del código del proyecto.

2. Requerimientos del Proyecto

2.1 SIA-1 — Análisis de datos y funcionalidades

El sistema fue diseñado para gestionar reservas asociadas a los distintos recursos disponibles en un parque. Para determinar la información necesaria se consideraron las operaciones que debe realizar el sistema y la relación existente entre usuarios, recursos y reservas.

Los usuarios necesitan ser identificados de forma individual, por lo que se almacena su nombre y RUT. El RUT se utiliza como identificador para facilitar su búsqueda dentro del sistema.

Los recursos corresponden a los elementos que pueden ser reservados. Cada recurso posee un nombre, un identificador y una capacidad máxima. La capacidad es necesaria para impedir que se realicen reservas para una cantidad de personas superior a la permitida. Debido a que existen distintos tipos de recursos, se definió una clase abstracta Recurso y tres especializaciones: Cabana, Camping y Actividad. Cada una agrega información propia, como cantidad de habitaciones, disponibilidad de piscina, guía o duración de una actividad.

Una reserva relaciona a un usuario con un recurso. Para cada reserva se almacena un identificador, la cantidad de personas, el recurso reservado, una tarifa, un estado y un permiso asociado. Esta separación permite que un mismo usuario pueda mantener múltiples reservas y que cada una sea administrada individualmente.

La tarifa permite calcular el costo total a partir del precio por persona y la cantidad de integrantes de la reserva. Por otra parte, el permiso permite registrar si la reserva ha sido aprobada o continúa pendiente. Para organizar la información se utilizaron colecciones Map. Esta estructura fue seleccionada porque usuarios, recursos y reservas poseen identificadores únicos, por lo que pueden ser almacenados utilizando dichos identificadores como clave y posteriormente buscados directamente mediante ellos.

La organización principal de la información es que Parque guarda Usuarios, que guardan Reservas. Además, Parque mantiene la colección de recursos disponibles. De esta forma, cada usuario administra sus propias reservas y cada reserva referencia el recurso correspondiente.

A partir de este análisis se determinaron las principales funcionalidades del sistema: creación, búsqueda, modificación, eliminación y listado de usuarios; creación, búsqueda, modificación, cancelación y listado de reservas; búsqueda de recursos; verificación de capacidad; filtrado de recursos según cantidad de personas; cálculo de tarifas; y aprobación o rechazo de permisos.

Esta estructura busca que cada clase represente una responsabilidad concreta del problema y evita concentrar toda la información y lógica en una sola clase.

2.2 SIA-2 — Diseño conceptual de clases y código Java

El diseño del sistema se realizó utilizando clases que representan las principales entidades identificadas durante el análisis.

La clase Recurso fue definida como una clase abstracta que concentra los atributos comunes de todos los recursos: nombre, identificador y capacidad. Las clases Cabana, Camping y Actividad heredan de ella y agregan sus características particulares. Esta decisión evita repetir atributos y comportamiento común y permite utilizar polimorfismo al trabajar con objetos de tipo Recurso.

La clase Parque representa el contexto general del sistema y mantiene las colecciones de recursos y usuarios. La clase Usuario, a su vez, mantiene una colección de objetos Reserva, lo que representa que cada usuario puede poseer varias reservas.

Reserva relaciona al usuario con un Recurso y mantiene la información propia de la operación, como cantidad de personas, estado, tarifa y permiso. Tarifa concentra la lógica utilizada para calcular el precio de una reserva, mientras que Permiso administra su estado de aprobación.

Las relaciones principales del modelo son:

- Recurso es generalizado por Cabana, Camping y Actividad.
- Parque administra objetos Usuario y Recurso.
- Cada Usuario mantiene múltiples objetos Reserva.
- Cada Reserva se encuentra asociada a un Recurso, una Tarifa y un Permiso.
- MenuConsola y MenuVentana utilizan el mismo modelo de dominio, cambiando solamente la forma en que el usuario interactúa con el sistema.
- Lector y Escritor se encargan de la persistencia sin trasladar esa responsabilidad a las entidades del dominio.

El diagrama UML fue actualizado de acuerdo con la implementación final, incluyendo atributos, métodos y relaciones existentes entre las clases. Esto permite que el diseño conceptual represente efectivamente el código entregado y no una versión anterior del sistema.

2.3 SIA-3 — Buenas prácticas

El proyecto utiliza encapsulamiento manteniendo los atributos de las clases como private y proporcionando métodos get y set para controlar el acceso a ellos. Esto evita que los atributos sean modificados directamente desde otras clases y mantiene la responsabilidad de cada objeto sobre su propio estado.

También se aplicó una separación de responsabilidades. La clase Parque administra recursos y usuarios; Usuario administra las reservas que le pertenecen; Reserva mantiene la

información de cada reserva; Tarifa realiza los cálculos de precio; y Permiso administra el estado de aprobación.

La lectura y escritura de información fueron separadas del modelo mediante las clases Lector y Escritor. De esta manera, las entidades del sistema no necesitan encargarse directamente de manipular archivos CSV.

Del mismo modo, la interacción con el usuario fue separada en MenuConsola y MenuVentana. La clase Main se mantiene como punto de entrada del programa y se limita principalmente a cargar los datos, permitir la selección del modo de ejecución, iniciar el menú correspondiente y guardar la información al finalizar. Esto evita concentrar toda la lógica del programa en Main.

Para la navegación de los menús se utilizan estructuras switch, separando cada operación en métodos específicos. También se evitó depender de comprobaciones mediante instanceof para mostrar distintos tipos de recursos. En su lugar, se utiliza sobrescritura de métodos de Recurso, permitiendo que cada clase concreta determine cómo representar su propia información.

Se revisaron además los nombres de atributos y métodos para utilizar nombres más representativos y consistentes, por ejemplo capacidad, habitaciones, piscina, mostrarRecursosPorCapacidad y obtenerDescripcion.

La verificación de disponibilidad se encuentra centralizada en el método hayDisponibilidad(int), evitando repetir la misma validación en distintas partes del programa.

Finalmente, las principales clases y métodos cuentan con documentación Javadoc que describe su propósito, parámetros y valores retornados cuando corresponde. Esto facilita la lectura y mantenimiento del código y constituye además la implementación del requisito opcional SIA-O3.

Los datos iniciales del sistema no se encuentran escritos directamente en Main, sino que son cargados desde archivos mediante Lector, permitiendo mantener separada la información de la lógica del programa.

2.4 SIA-4 — Colecciones de objetos

El sistema utiliza colecciones de Java para almacenar y administrar los objetos principales. Se eligió principalmente la interfaz Map, ya que los elementos relevantes del sistema poseen identificadores que pueden utilizarse como claves para facilitar su búsqueda y administración.

En la clase Parque se mantienen dos colecciones:

- Map<String, Recurso> recursos
- Map<String, Usuario> usuarios

La primera permite almacenar los distintos recursos disponibles en el parque utilizando su identificador como clave. La segunda permite almacenar los usuarios utilizando su RUT como identificador. Además, cada objeto Usuario mantiene su propia colección “Map<String, Reserva> reservas”, donde cada reserva se encuentra asociada a su identificador correspondiente.

De esta forma se genera una estructura de colecciones anidadas:

Parque

```
└─ Map<String, Usuario>
    └─ Map<String, Reserva>
```

Esta estructura permite representar de manera directa la relación del problema: el parque contiene usuarios y cada usuario puede mantener múltiples reservas.

La utilización de Map permite además implementar de manera simple las operaciones de inserción, búsqueda, modificación, eliminación y listado sobre los objetos almacenados.

```
public class Parque {
    private String nombre;
    private String ubicacion;
    private Map<String, Recurso> recursos;
    private Map<String, Usuario> usuarios;
```

```
public class Usuario {
    private String nombre;
    private String rut;
    private Map<String, Reserva> reservas;
```

2.5 SIA-5 — Sobrecarga de métodos

El proyecto implementa sobrecarga de métodos en dos clases distintas: Usuario y Tarifa.

Primera clase: Usuario

La clase Usuario posee dos versiones del método realizarReserva:

- realizarReserva(Recurso recurso, int cantidad)
- realizarReserva(Recurso recurso, int cantidad, boolean permisoAprobado)

Ambos métodos poseen el mismo nombre, pero reciben una cantidad diferente de parámetros.

La primera versión permite crear una reserva utilizando el comportamiento normal del sistema, mientras que la segunda permite además indicar el estado inicial del permiso. Esta

última alternativa resulta útil, por ejemplo, durante la reconstrucción de reservas almacenadas previamente.

La versión de dos parámetros reutiliza la versión más completa, evitando duplicar lógica.

```
public boolean realizarReserva(Recurso recurso, int cantidad, boolean permisoAprobado) {  
    recurso.validarCapacidad(cantidad);  
    Reserva reserva = new Reserva(cantidad, recurso, new Tarifa(200), true);  
    reserva.getPermiso().setAprobado(permisoAprobado);  
    reservas.put(reserva.getId(), reserva);  
    return true;  
}
```

```
public Reserva buscarReserva(Recurso recurso) {  
    for (Reserva reserva : reservas.values()) {  
        if (reserva.getRecurso().getId().equals(recurso.getId())) {  
            return reserva;  
        }  
    }  
    return null;  
}
```

Segunda clase: Tarifa

La clase Tarifa implementa las siguientes versiones del método calcular:

- calcular(int cantidadPersonas)
- calcular(int cantidadPersonas, double descuento)

La primera calcula el precio normal de la reserva según el número de personas. La segunda permite realizar el mismo cálculo incorporando un porcentaje de descuento. Además, la primera versión reutiliza la segunda utilizando un descuento de 0.0, por lo que ambas implementaciones forman parte efectiva del funcionamiento del programa.

```
public double calcular(int cantidadPersonas, double descuento) {  
    double total = precioBase * cantidadPersonas;  
    return total - (total * descuento / 100.0);  
}
```

```
public double calcular(int cantidadPersonas) {  
    return calcular(cantidadPersonas, descuento: 0.0);  
}
```

2.6 SIA-6 — Sobreescritura de métodos

El sistema utiliza sobreescritura de métodos mediante la jerarquía formada por la clase abstracta Recurso y sus clases hijas Cabana, Camping y Actividad.

La clase Recurso define operaciones comunes que deben ser implementadas por cada tipo específico de recurso, entre ellas mostrarRecurso() y obtenerDescripcion().

Las tres clases hijas implementan su propia versión de estos métodos utilizando la anotación @Override.

Por ejemplo, una Cabana muestra su cantidad de habitaciones, un Camping indica si posee piscina y una Actividad incorpora información sobre el guía y la duración.

Esto permite trabajar con objetos mediante una referencia del tipo general Recurso sin que las demás clases tengan que determinar manualmente cuál es su tipo concreto.

Por ejemplo, el programa puede ejecutar recurso.mostrarRecurso(); y Java selecciona automáticamente la implementación correspondiente a Cabana, Camping o Actividad.

El mismo principio se utiliza con obtenerDescripcion(), lo que permitió reutilizar la lógica tanto en la interfaz de consola como en la interfaz gráfica y evitar estructuras basadas en múltiples comprobaciones instanceof.

```
/**
 * Muestra los datos del recurso por consola.
 */
public abstract void mostrarRecurso();

/**
 * Genera una descripción textual del recurso.
 *
 * @return descripción del recurso
 */
public abstract String obtenerDescripcion();
```

```
/**
 * Genera una descripción con los datos de la cabaña.
 *
 * @return descripción textual de la cabaña
 */
@Override
public String obtenerDescripcion() {
    return "Nombre: " + getNombre() + "\n" +
           "ID: " + getId() + "\n" +
           "Capacidad: " + getCapacidad() + " personas\n" +
           "Habitaciones: " + habitaciones;
}

@Override
public void mostrarRecurso(){
    System.out.println(obtenerDescripcion());
}
```

2.7 SIA-7 — Inserción y listado

El sistema permite insertar y listar elementos de las dos colecciones principales utilizadas para representar la relación anidada del proyecto: usuarios y reservas.

Inserción de usuarios

La clase Parque mantiene la colección de usuarios y permite agregar nuevos objetos mediante agregarUsuario(Usuario usuario).

Cuando el usuario ingresa un RUT que no se encuentra registrado, el sistema permite crear un nuevo objeto Usuario y posteriormente incorporarlo a la colección del parque.

Esta funcionalidad se encuentra disponible tanto desde MenuConsola como desde MenuVentana.

```
public boolean agregarUsuario(Usuario u) {  
    if (usuarios.containsKey(u.getRut())) {  
        return false;  
    }  
    usuarios.put(u.getRut(), u);  
    return true;  
}
```

Listado de usuarios

La colección de usuarios puede recorrerse para mostrar los usuarios actualmente registrados en el parque. El listado permite visualizar información como Nombre y RUT, y se encuentra disponible mediante las opciones de gestión de usuarios de ambas interfaces.

```
public void listarUsuarios() {  
    if (usuarios.isEmpty()) {  
        System.out.println("No hay usuarios registrados.");  
        return;  
    }  
  
    for (Usuario usuario : usuarios.values()) {  
        usuario.mostrarUsuario();  
        System.out.println("-----");  
    }  
}
```

Inserción de reservas

La segunda colección corresponde a las reservas almacenadas dentro de cada objeto Usuario.

Cuando se realiza una reserva, el sistema:

1. Busca el recurso solicitado.
2. Verifica la cantidad de personas.
3. Comprueba la capacidad del recurso.
4. Crea un objeto Reserva.
5. Lo agrega al Map de reservas del usuario.

La inserción se realiza mediante los métodos realizarReserva(...) de la clase Usuario. De esta forma, la reserva queda almacenada dentro de la colección anidada correspondiente al usuario que la realizó.

```

public boolean realizarReserva(Recurso recurso, int cantidad, boolean permisoAprobado) {
    recurso.validarCapacidad(cantidad);
    Reserva reserva = new Reserva(cantidad, recurso, new Tarifa(200), true);
    reserva.getPermiso().setAprobado(permisoAprobado);
    reservas.put(reserva.getId(), reserva);
    return true;
}

```

Listado de reservas

Cada usuario puede listar sus reservas mediante la colección “Map<String, Reserva> reservas”. El listado muestra información como ID de reserva, Cantidad de personas, Recurso asociado, Tarifa, Estado y Permiso. La operación se encuentra disponible tanto en consola como en ventanas.

```

public void listarReservas() {
    if (reservas.isEmpty()) {
        System.out.println("El usuario no tiene reservas.");
        return;
    }

    for (Reserva reserva : reservas.values()) {
        reserva.mostrarReserva();
        System.out.println("-----");
    }
}

```

2.8 SIA-8 — Edición, eliminación y búsqueda

El sistema implementa operaciones de búsqueda, modificación y eliminación sobre las dos colecciones utilizadas en la estructura anidada: usuarios y reservas.

Usuarios

Los usuarios pueden ser buscados mediante su RUT utilizando buscarUsuario(String rut). Esta búsqueda aprovecha que el RUT es utilizado como clave de la colección Map<String, Usuario>.

El sistema también permite modificar la información de un usuario, específicamente su nombre, mediante setNombre(String nombre). La eliminación se realiza mediante una operación que retira al usuario de la colección mantenida por Parque. Estas funcionalidades se encuentran disponibles desde el menú de gestión de usuarios.

```

public Usuario buscarUsuario(String rut) {
    return usuarios.get(rut);
}

```

```

public void setNombre(String nombre) {
    this.nombre = nombre;
}

```

Reservas

Las reservas pueden buscarse utilizando su identificador mediante `buscarReserva(String id)`. Cada reserva puede modificar la cantidad de personas mediante `cambiarCantidad(int nuevaCantidad)`.

Antes de realizar el cambio, el sistema comprueba que la nueva cantidad sea válida y no supere la capacidad del recurso asociado. También es posible cancelar o eliminar una reserva utilizando su identificador. Además, las reservas permiten administrar el permiso asociado mediante operaciones de aprobación y rechazo.

```
public Reserva buscarReserva(String id) {  
    return reservas.get(id);  
}
```

```
public boolean cambiarCantidad(int nuevaCantidad) {  
    if (nuevaCantidad <= 0) {  
        return false;  
    }  
    recurso.validarCapacidad(nuevaCantidad);  
    cantidad = nuevaCantidad;  
    return true;  
}
```

2.9 SIA-9 — Funcionalidad propia de utilidad para el negocio

Como funcionalidad propia del negocio se implementó un filtro que permite obtener los recursos adecuados para una cantidad determinada de personas.

La funcionalidad principal se encuentra en la clase `Parque` mediante `filtrarRecursosPorCapacidad(int cantidadPersonas)`.

El método recorre los recursos registrados y utiliza `recurso.hayDisponibilidad(cantidadPersonas)` para comprobar si cada recurso puede recibir al grupo solicitado.

Los recursos que cumplen el criterio se agregan a una nueva colección `List<Recurso>`. Por lo tanto, el método no solo muestra información, sino que genera realmente un subconjunto filtrado de la colección original. El proceso puede representarse de la siguiente forma:

1. Colección completa de recursos
2. Cantidad solicitada por el usuario
3. `hayDisponibilidad(cantidad)`
4. `List<Recurso>` con recursos compatibles

El criterio considera que la cantidad ingresada sea válida y que no supere la capacidad máxima del recurso.

La funcionalidad resulta útil para el negocio porque permite encontrar rápidamente cabañas, campings o actividades capaces de recibir a un grupo determinado, evitando que el usuario tenga que revisar manualmente todos los recursos disponibles.

```
public List<Recurso> filtrarRecursosPorCapacidad(int cantidadPersonas) {  
    List<Recurso> recursosDisponibles = new ArrayList<Recurso>();  
  
    for (Recurso recurso : recursos.values()) {  
        if (recurso.hayDisponibilidad(cantidadPersonas)) {  
            recursosDisponibles.add(recurso);  
        }  
    }  
  
    return recursosDisponibles;  
}
```

```
public boolean hayDisponibilidad(int cantidad) {  
    return cantidad <= capacidad && cantidad > 0;  
}
```

2.10 SIA-10 — Consola y ventanas

El sistema permite seleccionar al inicio entre dos modalidades de interacción: interfaz mediante consola e interfaz gráfica mediante ventanas.

La modalidad de consola se encuentra implementada en la clase MenuConsola, mientras que la modalidad gráfica se encuentra implementada mediante MenuVentana, utilizando componentes de Java Swing.

La selección del modo se realiza desde Main, después de cargar los datos del sistema. Ambas interfaces trabajan sobre el mismo objeto Parque, por lo que utilizan la misma información y lógica de negocio.

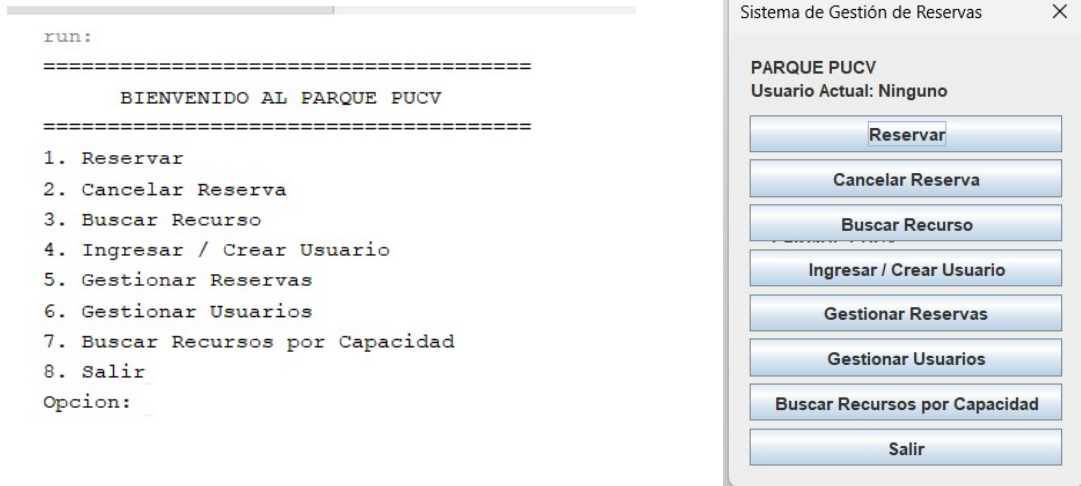
Las dos modalidades permiten acceder a las principales funcionalidades:

- Realizar reservas.
- Cancelar reservas.
- Buscar recursos.
- Ingresar o crear usuarios.
- Gestionar reservas.
- Gestionar usuarios.
- Buscar recursos según capacidad.

- Salir del sistema.

La interfaz gráfica utiliza componentes como JDialog, JPanel, JButton, JTextArea, JScrollPane y JOptionPane. Los menús fueron organizados verticalmente y los listados extensos utilizan barras de desplazamiento para evitar que la información exceda el tamaño de la pantalla.

De esta manera, el usuario puede realizar las mismas operaciones principales independientemente del modo de interacción elegido.



2.11 SIA-11 — Persistencia de datos

El sistema implementa persistencia mediante archivos CSV, permitiendo conservar información entre distintas ejecuciones del programa.

La responsabilidad de persistencia se separó en dos clases:

- Lector: responsable de cargar los datos.
- Escritor: responsable de guardar los cambios.

Al iniciar la aplicación, Main utiliza Lector para cargar la información del parque, los usuarios y las reservas.

Entre los métodos utilizados se encuentran leerParque(...), leerUsuarios(...) y leerReservas(...). Las reservas recuperan información como el usuario asociado, recurso reservado, cantidad de personas y estado del permiso.

Durante la ejecución, los usuarios pueden crear o modificar información. Al seleccionar la opción de salir, el sistema utiliza Escritor para guardar nuevamente el estado actualizado. Entre los métodos utilizados se encuentran guardarUsuarios(...) y guardarReservas(...).

Por lo tanto, la persistencia funciona mediante un proceso batch:

1. Inicio
2. Lectura de archivos CSV
3. Ejecución del sistema
4. Modificaciones realizadas por el usuario
5. Salida
6. Escritura de los datos actualizados

Esto permite, por ejemplo, aprobar un permiso, cerrar el programa y conservar ese estado al ejecutar nuevamente la aplicación.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileNotFoundException;
import java.io.IOException;
/**
 * Responsable de cargar desde archivos CSV los datos iniciales
 * utilizados por el sistema.
 */
public class Lector {

    public static Parque leerParque(String nombreArchivo) {
        try {
            BufferedReader archivo = new BufferedReader(new FileReader(nombreArchivo));
            Parque parque = null;
            String linea;

            while ((linea = archivo.readLine()) != null) {
                linea = linea.trim();

                if (linea.equals("") || linea.startsWith("#")) {
                    continue;
                }
                String[] datos = linea.split(";");
                if (datos[0].equalsIgnoreCase("tipo")) {
                    continue;
                }
                if (datos[0].equalsIgnoreCase("PARQUE")) {
                    parque = new Parque(datos[1], datos[4]);
                }
                else if (datos[0].equalsIgnoreCase("CABANA")) {
                    String nombre = datos[1];

```

```
tipo;nombre;id;capacidad;dato1;dato2
PARQUE;Parque PUCV;;;Valparaiso;
CABANA;Bosque Norte;CAB1;6;3;
CABANA;Mirador Sur;CAB2;4;2;
CABANA;Costa Azul;CAB3;8;4;
CAMPING;Camping A;CAM1;6;true;
CAMPING;Camping B;CAM2;4;false;
CAMPING;Camping C;CAM3;8;true;
ACTIVIDAD;Senderismo;ACT1;10;Juan Perez;70
ACTIVIDAD;Kayak;ACT2;8;Maria Soto;70
ACTIVIDAD;Observacion de Aves;ACT3;12;Pedro Rojas;60
```

```

public static void guardarUsuarios(String nombreArchivo, Parque parque) throws IOException {
    try (PrintWriter archivo = new PrintWriter(new FileWriter(nombreArchivo))) {
        archivo.println("tipo;nombre;rut");
        for (Usuario usuario : parque.getUsuarios()) {
            archivo.println("USUARIO;" + usuario.getNombre() + ";" + usuario.getRut());
        }
    }
}

```

2.12 SIA-12 — Excepciones personalizadas

El proyecto implementa dos excepciones personalizadas para representar errores propios de la lógica del sistema `CapacidadExcedidaException` y `ReservaNoEncontradaException`.

CapacidadExcedidaException

Esta excepción representa el intento de utilizar una cantidad de personas superior a la capacidad permitida por un recurso.

La validación se encuentra relacionada con la lógica de los recursos y reservas. Cuando la cantidad solicitada supera la capacidad máxima, se lanza `CapacidadExcedidaException`.

La excepción es utilizada en operaciones como:

- creación de una reserva;
- modificación de la cantidad de personas de una reserva.

La excepción es capturada mediante bloques try-catch en las interfaces, permitiendo informar el problema al usuario sin finalizar abruptamente la ejecución.

ReservaNoEncontradaException

Esta excepción representa la situación en que se intenta realizar una operación sobre una reserva cuyo identificador no existe.

Por ejemplo, al intentar cancelar una reserva inexistente, el sistema lanza `ReservaNoEncontradaException`, la excepción también es capturada mediante try-catch, mostrando un mensaje controlado al usuario.

El flujo general de manejo de excepciones es:

1. Operación
2. Validación
3. throw excepción personalizada
4. try-catch en la interfaz
5. Mensaje al usuario

```

public boolean cancelarReserva(String id)
    throws ReservaNoEncontradaException {

    if (!reservas.containsKey(id)) {
        throw new ReservaNoEncontradaException(id);
    }

    reservas.remove(id);
    return true;
}

```

2.13 SIA-13 — GitHub

El proyecto utiliza GitHub como sistema de control de versiones durante todo el desarrollo. El repositorio permitió registrar progresivamente la creación de clases, implementación de funcionalidades, correcciones y mejoras realizadas durante el proyecto.

El historial supera ampliamente el mínimo de seis commits solicitado. Entre los cambios registrados se encuentran commits relacionados con:

- inicialización de las clases;
- implementación y posterior simplificación de Main;
- separación del menú de consola;
- lectura y persistencia de datos;
- gestión de usuarios y reservas;
- implementación de la interfaz gráfica;
- mejoras de encapsulamiento;
- implementación de sobrecarga y filtrado;
- incorporación de Javadoc;
- creación e integración de excepciones personalizadas.

Algunos commits representativos del desarrollo son:

- Separar menú de consola de la clase Main
- Completar gestión de usuarios reservas tarifas y permisos
- Agregar persistencia y gestión completa de reservas
- Agregar interfaz ventana y selección de modo
- Mejorar encapsulamiento y documentación del sistema
- Completar sobrecarga y filtro de recursos
- Agregar documentación Javadoc al sistema
- Creación e implementación de excepciones
- Corregir integración de excepciones y mantener persistencia

El uso de commits permitió conservar distintas etapas del proyecto, integrar el trabajo realizado por los integrantes y registrar las correcciones efectuadas a partir del feedback recibido.

The screenshot displays the GitHub interface for the repository 'joacopizarroguvcv / SIA-Sistema-Gestion-Reserva-PQ'. The main section shows a list of files and their corresponding commit messages and timestamps. The files are organized in a table with columns for the file name, the commit message, and the time since the last commit. The commit messages are in Spanish, describing updates, documentation additions, and exception handling. The page also shows repository statistics like 40 commits, 0 stars, and 0 forks.

File	Commit Message	Time
.gitignore	Actualización de clases + agregación de metodos para parque...	3 weeks ago
Actividad.java	Agregar documentacion Javadoc al sistema	2 hours ago
Cabana.java	Agregar documentacion Javadoc al sistema	2 hours ago
Camping.java	Agregar documentacion Javadoc al sistema	2 hours ago
CapacidadExcedidaException.java	creación e implementación de excepciones	1 hour ago
Escritor.java	Mejorar encapsulamiento y documentacion del sistema	2 hours ago
Lector.java	Corregir integracion de excepciones y mantener persistencia	1 hour ago
Main.java	Mejorar encapsulamiento y documentacion del sistema	2 hours ago
MenuConsola.java	Corregir integracion de excepciones y mantener persistencia	1 hour ago
MenuVentana.java	Corregir integracion de excepciones y mantener persistencia	1 hour ago
Parque.java	Completar sobrecarga y filtro de recursos	2 hours ago
Permiso.java	Agregar documentacion Javadoc al sistema	2 hours ago
README.md	Create README.md	3 weeks ago
Recurso.java	Corregir integracion de excepciones y mantener persistencia	1 hour ago
Reserva.java	Corregir integracion de excepciones y mantener persistencia	1 hour ago
ReservaNoEncontradaException.java	Corregir integracion de excepciones y mantener persistencia	1 hour ago
Tarifa.java	Agregar documentacion Javadoc al sistema	2 hours ago
Usuario.java	Corregir integracion de excepciones y mantener persistencia	1 hour ago
parque.csv	Actualización de las clases, en donde cambie los mapas e Impl...	20 hours ago
reservas.csv	Agregar persistencia y gestion completa de reservas (la anteri...	5 hours ago
usuario.csv	Agregar persistencia y gestion completa de reservas (la anteri...	5 hours ago

Enlace al repositorio:

<https://github.com/joacopizarroguvcv/SIA-Sistema-Gestion-Reserva-PQ>

2.14 SIA-O3 — Documentación mediante Javadoc

Como requisito opcional se incorporó documentación Javadoc en las principales clases y métodos del proyecto. La documentación describe el propósito de las clases, constructores y métodos relevantes, incluyendo parámetros y valores retornados cuando corresponde.

Se documentaron las principales clases del sistema y los métodos relevantes relacionados con disponibilidad, reservas, filtrado, persistencia y cálculo de tarifas.

```
/**
 * Crea un nuevo camping.
 *
 * @param nombre nombre del camping
 * @param id identificador del recurso
 * @param capacidad cantidad máxima de personas
 * @param piscina indica si posee piscina
 */
```

```
/**
 * Guarda los usuarios actualmente registrados en el parque.
 *
 * @param nombreArchivo archivo CSV de destino
 * @param parque parque cuyos usuarios serán almacenados
 * @throws IOException si ocurre un error durante la escritura
 */
```

3. Conclusión

El proyecto permitió desarrollar un sistema de gestión de reservas para un parque utilizando programación orientada a objetos. La solución final administra usuarios, recursos, reservas, tarifas y permisos, utilizando clases con responsabilidades diferenciadas y colecciones para organizar la información.

Durante el desarrollo se aplicaron conceptos como encapsulamiento, herencia, polimorfismo, sobrecarga, sobrescritura, colecciones anidadas, excepciones personalizadas, persistencia y desarrollo de interfaces mediante consola y Swing.

El sistema permite administrar usuarios y reservas mediante operaciones de creación, búsqueda, modificación, eliminación y listado. También permite buscar recursos, verificar su capacidad, filtrar recursos según una cantidad determinada de personas, calcular tarifas y gestionar la aprobación de permisos. Los datos pueden mantenerse entre ejecuciones mediante archivos CSV y las funcionalidades se encuentran disponibles tanto en consola como en una interfaz gráfica.

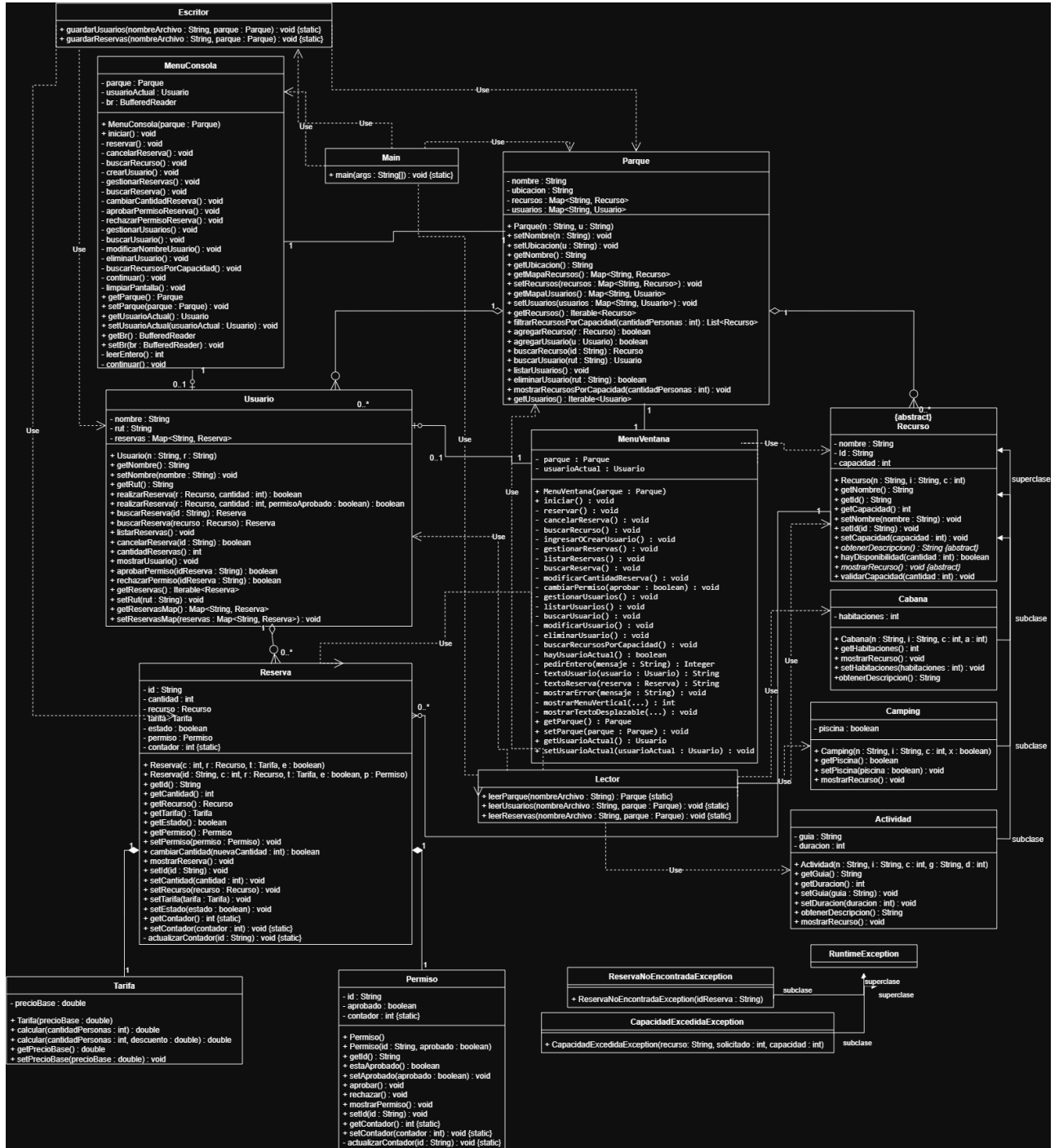
Una de las principales dificultades fue organizar correctamente las responsabilidades de las clases y evitar concentrar lógica en Main. También fue necesario ajustar la estructura de colecciones para representar adecuadamente la relación entre usuarios y reservas. Durante la integración de los distintos componentes también se presentaron dificultades relacionadas con persistencia, validaciones, identificadores y manejo de excepciones. Estas situaciones permitieron revisar y mejorar el diseño inicial.

El proyecto permitió aplicar conceptos de programación orientada a objetos en un sistema de mayor tamaño que los ejercicios individuales realizados durante el curso. En particular, se reforzó la importancia de separar responsabilidades, utilizar colecciones adecuadas, mantener encapsulados los datos, reutilizar comportamiento mediante herencia y polimorfismo, gestionar errores mediante excepciones y mantener un historial de desarrollo mediante GitHub.

Como mejoras futuras se podría incorporar una interfaz gráfica de mayor complejidad, agregar fechas y horarios a las reservas, implementar distintos valores de tarifa según el recurso y utilizar una base de datos en lugar de archivos CSV.

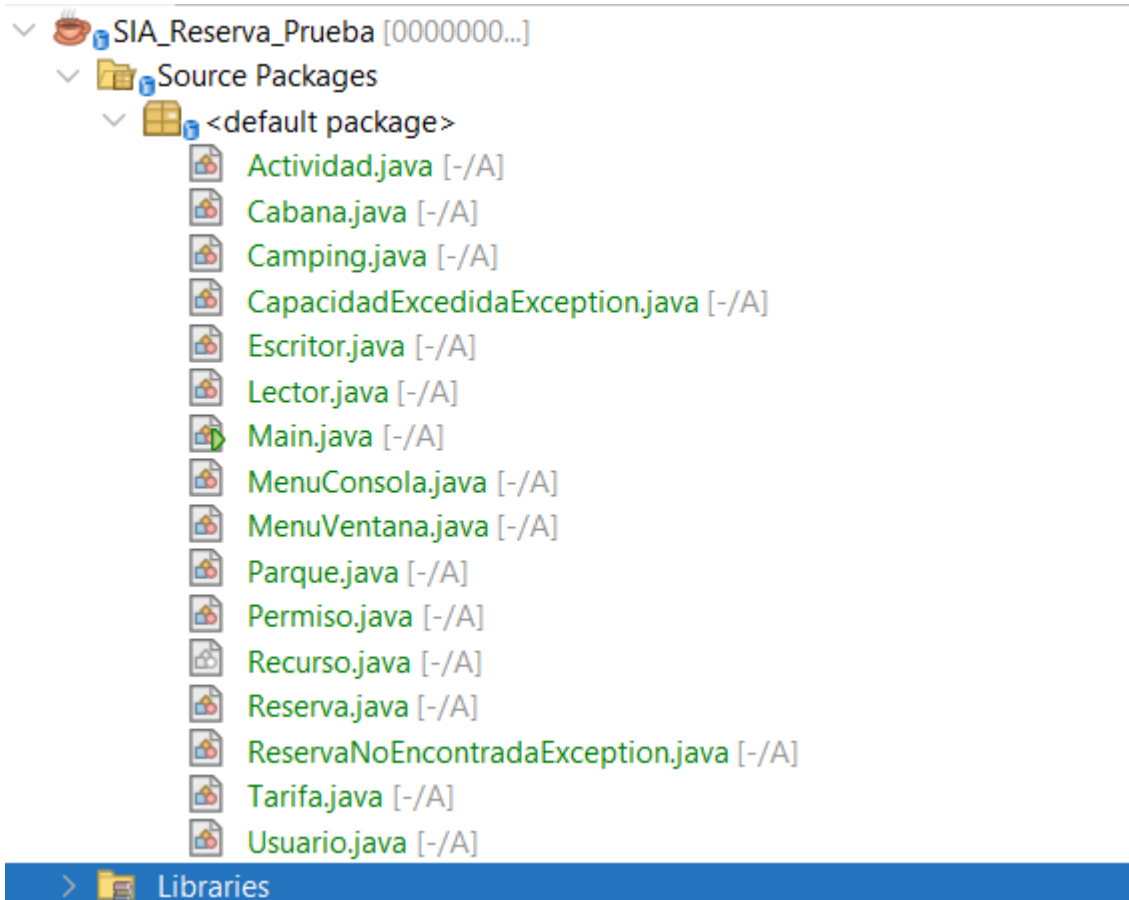
4. Anexos

Diagrama UML completo



Estructura de carpetas

Se incluye una captura de la estructura de carpetas del proyecto, mostrando las clases Java y los archivos utilizados para la persistencia. La versión final del proyecto fue importada, compilada y ejecutada correctamente en NetBeans.



Archivos de datos utilizados

tipo;nombre;id;capacidad;dato1;dato2		
PARQUE;Parque PUCV;;;Valparaiso;		
CABANA;Bosque Norte;CAB1;6;3;		
CABANA;Mirador Sur;CAB2;4;2;		
CABANA;Costa Azul;CAB3;8;4;		
CAMPING;Camping A;CAM1;6;true;		
CAMPING;Camping B;CAM2;4;false;		
CAMPING;Camping C;CAM3;8;true;		
ACTIVIDAD;Senderismo;ACT1;10;Juan Perez;70		
ACTIVIDAD;Kayak;ACT2;8;Maria Soto;70		
ACTIVIDAD;Observacion de Aves;ACT3;12;Pedro Rojas;60		

tipo;nombre;rut	
USUARIO;Juan Perez;98765432-1	
USUARIO;Maria Soto;11222333-4	
USUARIO;Prueba;11111111-1	
USUARIO;Daniel Robledo;12345678-9	

rut;idRecurso;cantidad;permisoAprobado		
98765432-1;CAM1;2;false		
11222333-4;ACT1;1;false		
12345678-9;ACT2;2;false		
12345678-9;CAB1;3;true		
12345678-9;CAB1;3;true		
12345678-9;ACT1;2;false		